Distributed Applications & Web Services

Name: Emmanuel Manu
Student ID: 100862618

Contents

1. Introduction 2

2. Application structure and HTTP API 2

3. Local state and metadata 3

4. TCP and UDP communication 3

5. Remote file metadata with gRPC 4

6. Testing, limitations and conclusion 5

Conclusion 5

References 6

1. Introduction

This coursework involved developing one Go application over five weeks, adding new features while making sure the earlier work is also working. A distributed application is a program that runs on one or more computers simultaneously and communicates through a network ‘Couchbase (2026). I started with just a basic node and then I gradually added HTTP resources, state management, TCP and UDP communication, and also a gRPC service for remote file metadata.

An important idea throughout the project was that two nodes can run the same code and without sharing the same memory. Each node has its own identity and configuration and local data. Information about another node helps a process address it, but it does not automatically copy the state of that node.

2. Application structure and HTTP API

In Week 1, I implemented and tested the basic node setup, configuration validation, startup behaviour and /healthz endpoint. I also learned to distinguish a node's logical identity from its network address. The identity also tells us which participant is running, whereas the address identifies where a service can be reached. The health endpoint is a liveness check and is not proof that every other service or node is available.

In Week 2, I added the status resource and also request validation. The /v1/status endpoint represents the application's simulated status, while /healthz continues to report whether the HTTP service responds. Keeping all of these purposes separate avoids treating an application status value as a liveness measurement. Request checks for the methods, content types, JSON input and body size also help the API to reject invalid requests consistently rather than accepting data that is malformed.

3. Local state and metadata

Week 3 also introduced the peer registry and file catalogue. The registry stores peer records, and the catalogue stores the file metadata. They do not contain the actual file contents. This distinction helped me understand that a metadata service is not the same as a distributed file-storage system.

The state components needed to validate records and provide results that are predictable. Duplicate peer records were rejected, and snapshots were also returned independently and in a consistent order. This is important because callers should not be able to accidentally change the internal state by modifying a returned slice. The implementation also uses locking around shared state so that the concurrent operations do not simply read and write the same collections without coordination. I tested the duplicate-peer behaviour and also the registry and catalogue operations.

4. TCP and UDP communication

In Week 4, I extended the nodes to exchange messages over TCP and UDP and connect incoming messages to the local inbox. The transports behave differently. TCP provides an ordered byte stream, so the application must define message framing to differentiate between one message from another. UDP preserves datagram boundaries, but also a datagram can be lost or fail to reach the receiver. TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are two core protocols of the Transport Layer of the OSI and TCP/IP models *'GeekforGeeks' (2026)*.

Validation checks helped me reject malformed input, while size limits and deadlines also helped prevent a request from waiting indefinitely or accepting an unnecessarily large message. The acknowledgement reports whether the receiving application accepted a message and it should not be treated as authentication of the sender. The two-node tests checked communication through the network-facing code rather than shared in-memory data.

5. Remote file metadata with gRPC

In Week 5, I added gRPC and Protocol Buffers for remote file-metadata requests. gRPC is an open source, language agnostic and cross-platform remote procedure call (RPC) framework that uses HTTP/2 transport layer protocol ‘IBM’ (2026).

In this project the .proto file defines the service contract and also message fields. The generated Go files provide the client and server interfaces, while the handwritten service code also implements the behaviour.

The ListFiles service returns metadata from the requested node's catalogue. The client uses a bounded context, and the service validates the request before returning data. An empty successful catalogue must be distinguished from a failed remote call, because an empty list is a valid result while an unavailable service means that no valid result was received.

I connected the remote call to the HTTP API and through /v1/remote-files. During the demonstration of both two nodes, Node A requested metadata from Node B and received diagram.txt and notes.txt, rather than its own alpha.txt. I also tested the reverse direction, with Node B requesting Node A's metadata. The different fixture files helped show that the result came from the remote catalogue and not a hard-coded local response.

I then stopped Node B while Node A was still running. The remote request through Node A returned HTTP 503 with remote_unavailable, while Node A's /healthz and local /v1/files endpoints continued to respond. After B restarted, the remote request succeeded again. This mapped the remote failure clearly without making the whole local node appear unavailable.

6. Testing, limitations and conclusion

I ran the full Go test suite, build and vet checks, module verification, and focused tests for remote listing, cancellation, unavailable services, invalid requests, empty catalogues and HTTP error mapping. The following commands go test ./... -count=1 -timeout 20s, go build ./..., go vet ./... and go mod verify completed successfully.

The race-enabled suite did not run in my environment because CGO_ENABLED was set to 0, and go test -race ./... reported that the race detector requires cgo. I treated race testing as a check and not a passed test. Ordinary tests are useful evidence, but they do not establish that all concurrency cases are race-free.

Conclusion

This project helped me understand that distributed applications require more than opening a network connection and that each component must validate input, keep local and remote state distinct, handle failures and report results honestly

References

'Couchbase' (2026) Available at: https://www.couchbase.com/blog/distributed-applications (Accessed: 23 October 2026).

'GeekforGeeks' (2026) Available at: https://www.geeksforgeeks.org/computer-networks/differences-between-tcp-and-udp (Accessed: 23 October 2026).

'IBM' (2026) Available at: https://www.ibm.com/think/topics/grpc (Accessed: 23 October 2026).